

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»**

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)

09.03.01 ИНФОРМАТИКА И ВЫЧИСЛИТЕЛЬНАЯ ТЕХНИКА

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ

Кейс № 3

**Тема: Разработка веб-приложения «Kanban-доска» с системой
разграничения прав доступа и аудитом действий**

Руководитель: Чумакова М. Ю.

Студент: Каверина М.П.

Группа: БИВТ-24-3

Москва 2026

Содержание

1. ВВЕДЕНИЕ И ПРЕДМЕТНАЯ ОБЛАСТЬ	3
2. АНАЛИЗ ТРЕБОВАНИЙ И ЭТАПЫ РАЗРАБОТКИ.....	3
3. СТРУКТУРА БАЗЫ ДАННЫХ И ER-ДИАГРАММА.....	4
4. АРХИТЕКТУРА АРІ И ПРИМЕРЫ SQL-ЗАПРОСОВ	5
5. МОДЕЛЬ АВТОРИЗАЦИИ И БЕЗОПАСНОСТИ	9
6. РОЛИ И ПРАВА ДОСТУПА (RBAC).....	9
7. МЕХАНИЗМ АУДИТА И ЗАЩИТА ОТ КОЛЛИЗИЙ.....	10
8. ЗАКЛЮЧЕНИЕ	11
ИСТОЧНИКИ.....	13
Приложение А. Инструкция по запуску проекта	14

1. ВВЕДЕНИЕ И ПРЕДМЕТНАЯ ОБЛАСТЬ

Цель практики: Разработка полнофункционального MVP веб-приложения «Kanban-доска» (аналог Trello/Kaiten) для визуального управления задачами, с реализацией безопасной авторизации, ролевой модели доступа и журналирования действий пользователей.

Предметная область: Kanban-методология предполагает визуализацию рабочего процесса через перемещение карточек (задач) по колонкам (этапам). Система должна обеспечивать совместную работу, назначенных исполнителей, дедлайны и прозрачность изменений.

2. АНАЛИЗ ТРЕБОВАНИЙ И ЭТАПЫ РАЗРАБОТКИ

- Функциональные требования (MVP):
- Регистрация и аутентификация пользователей.
- CRUD-операции для досок, колонок и карточек.
- Drag-and-drop перемещение карточек между колонками.
- Система комментариев к карточкам.
- Разграничение прав доступа.
- Ведение журнала аудита (Audit Log) всех изменений.

Выбранный стек технологий:

- Backend: Python, FastAPI (асинхронный фреймворк, автоматическая документация OpenAPI).
- База данных: PostgreSQL.
- Frontend: HTML5, JavaScript, Tailwind CSS, библиотека SortableJS (для drag-and-drop).

Этапы разработки:

1. Проектирование ER-модели и схемы БД.
2. Реализация моделей данных и миграций.
3. Разработка API endpoints и логики авторизации (JWT + bcrypt).
4. Создание интерактивного пользовательского интерфейса.
5. Тестирование сценариев и написание документации.

3. СТРУКТУРА БАЗЫ ДАННЫХ И ER-ДИАГРАММА

База данных спроектирована с соблюдением принципов нормализации.

Все связи имеют корректные первичные (PK) и внешние (FK) ключи.

Основные сущности и связи (1:N):

users (PK: id) – хранит логин, хэш пароля и роль.

boards (PK: id, FK: owner_id => users.id) – доски задач.

columns (PK: id, FK: board_id => boards.id) – колонки на доске.

cards (PK: id, FK: column_id => columns.id) – карточки с полями title, description, priority, assignee, deadline, version.

comments (PK: id, FK: card_id => cards.id, FK: user_id => users.id) – комментарии.

audit_log (PK: id, FK: user_id => users.id) – история изменений. Связь с пользователем обязательна для отслеживания автора действия.

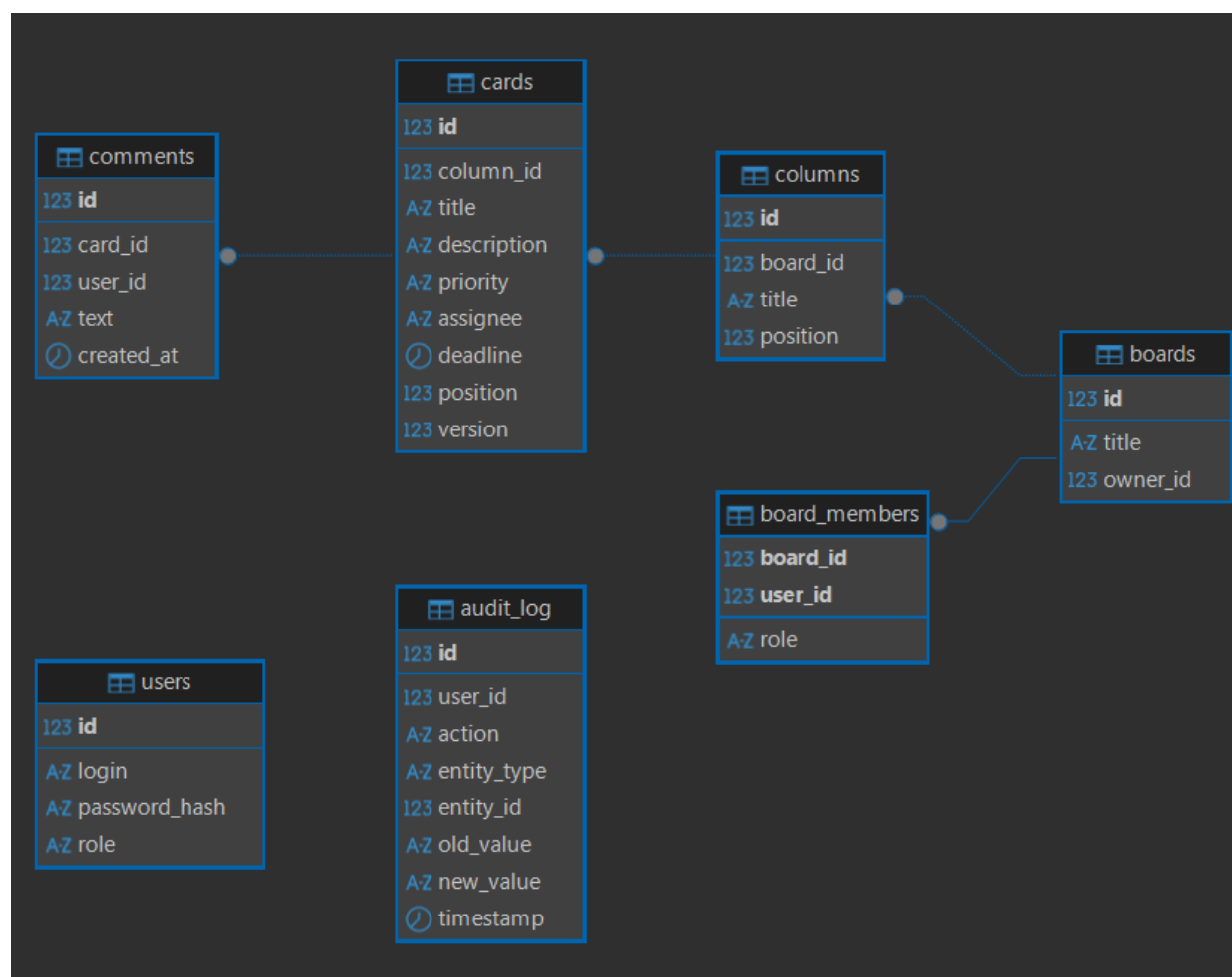


Рисунок 1. ER-диаграмма базы данных Канбан-приложения

4. АРХИТЕКТУРА API И ПРИМЕРЫ SQL-ЗАПРОСОВ

Backend реализован по принципу REST API. FastAPI автоматически генерирует интерактивную документацию OpenAPI (Swagger), где представлены все доступные endpoints, модели данных и схемы запросов/ответов. Для выполнения требований по сложности запросов (не только SELECT *) в проекте используются JOIN и GROUP BY.

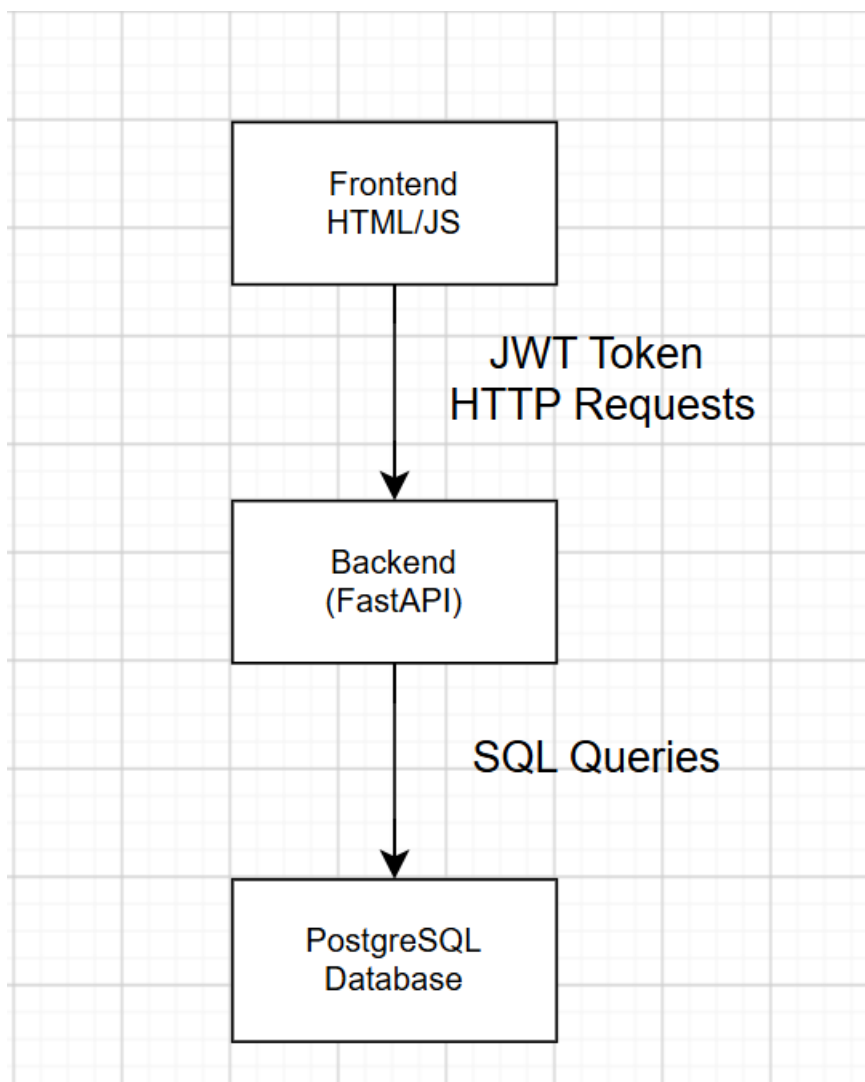


Рисунок 2. Архитектура взаимодействия компонентов приложения

Примеры SQL-запросов:

Пример 1: Запрос с JOIN для получения карточек доски с учетом порядка колонок:

```
SELECT c.id, c.title, c.description, c.priority, c.assignee, c.deadline, c.position, c.column_id
FROM cards c
JOIN columns col ON c.column_id = col.id
WHERE col.board_id = :board_id
ORDER BY col.position, c.position
```

Рисунок 3

Пример 2: Запрос с GROUP BY для получения статистики по приоритетам:

```
"SELECT c.priority, COUNT(c.id) as card_count FROM cards c GROUP BY c.priority"
```

Рисунок 4

Основные API Endpoints:

Таблица 1

Метод	Endpoint	Описание	Доступ
POST	/register	Регистрация нового пользователя	Public
POST	/login	Аутентификация, получение JWT токена	Public
GET	/users	Получение списка пользователей	Auth
PUT	/users/{user_id}/role	Изменение роли пользователя	Owner
GET	/boards	Получение списка досок	Auth
POST	/boards	Создание новой доски	Writer
PUT	/boards/{board_id}	Обновление доски	Writer
DELETE	/boards/{board_id}	Удаление доски	Writer
GET	/boards/{board_id}/columns	Получение колонок доски	Auth
DELETE	/columns/{column_id}	Удаление колонки	Writer
GET	/boards/{board_id}/cards	Получение карточек доски	Auth

POST	/cards	Создание карточки	Writer
PUT	/cards/{card_id}	Обновление карточки	Writer
PUT	/cards/{card_id}/move	Перемещение карточки	Writer
DELETE	/cards/{card_id}	Удаление карточки	Writer
POST	/cards/{card_id}/comments	Добавление комментария	Writer
GET	/audit-log	Получение журнала аудита	Auth
GET	/stats	Статистика по карточкам	Auth

default ^

GET	/	Root	✓
POST	/register	Register	✓
POST	/login	Login	✓
GET	/users	Get Users	✓
PUT	/users/{user_id}/role	Update User Role	✓
GET	/boards	Get Boards	✓
POST	/boards	Create Board	✓
PUT	/boards/{board_id}	Update Board	✓
DELETE	/boards/{board_id}	Delete Board	✓
GET	/boards/{board_id}/columns	Get Columns	✓
DELETE	/columns/{column_id}	Delete Column	✓
GET	/boards/{board_id}/cards	Get Cards	✓
POST	/cards	Create Card	✓
PUT	/cards/{card_id}	Update Card	✓
DELETE	/cards/{card_id}	Delete Card	✓
PUT	/cards/{card_id}/move	Move Card	✓
POST	/cards/{card_id}/comments	Add Comment	✓
GET	/cards/{card_id}/comments	Get Comments	✓
GET	/audit-log	Get Audit Log	✓
GET	/stats	Get Stats	✓

Рисунок 5. Swagger документация API: список всех доступных endpoints

Schemas		
BoardCreate	>	Expand all object
BoardUpdate	>	Expand all object
CardCreate	>	Expand all object
CardMove	>	Expand all object
CardUpdate	>	Expand all object
CommentCreate	>	Expand all object
HTTPValidationError	>	Expand all object
LoginRequest	>	Expand all object
UserCreate	>	Expand all object
UserUpdateRole	>	Expand all object
ValidationError	>	Expand all object

Рисунок 6. Модели данных (Pydantic schemas) для валидации запросов и ответов

5. МОДЕЛЬ АВТОРИЗАЦИИ И БЕЗОПАСНОСТИ

Авторизация реализована на основе JWT (JSON Web Token).

1. При регистрации/входе пароль не хранится в открытом виде. Используется библиотека passlib с алгоритмом bcrypt, который автоматически генерирует соль и создает криптографический хэш.
2. При успешном входе сервер генерирует JWT, содержащий sub (ID пользователя) и exp (время истечения: 60 минут).
3. Токен подписывается секретным ключом (SECRET_KEY). Клиент передает его в заголовке Authorization: Bearer <token>.
4. Зависимость `get_current_user` декодирует токен, проверяет подпись и извлекает пользователя из БД.

	123 id	AZ login	AZ password_hash	AZ role
1	1	admin	\$2b\$12\$t500w9sz06zibfceWp8HfelSrSoaxBz0Nzh/xhAj4WqKbzAZuG0Gu	OWNER

Рисунок 7. Шифрование пароля пользователя

6. РОЛИ И ПРАВА ДОСТУПА (RBAC)

Для управления доступом выбрана модель RBAC (Role-Based Access Control).

Обоснование выбора: В отличие от ABAC (Attribute-Based), RBAC проще в реализации, нагляднее и полностью покрывает требования MVP, обеспечивая четкое разделение обязанностей без избыточной сложности.

Реализован принцип минимальных привилегий:

Таблица 2

Роль	Доступ к чтению	Доступ к редактированию	Управление ролями
Читатель (reader)	Да	Нет	Нет
Писатель (writer)	Да	Да	Нет
Владелец (owner)	Да	Да	Да

В коде это обеспечивается декоратором-зависимостью `require_writer`, который проверяет роль текущего пользователя перед выполнением мутаций (POST, PUT, DELETE).

7. МЕХАНИЗМ АУДИТА И ЗАЩИТА ОТ КОЛЛИЗИЙ

Audit Log (История изменений):

Реализована функция `log_action`, которая фиксирует каждое изменение (CREATE, UPDATE, DELETE). В таблицу `audit_log` записываются: `user_id` (кто сделал), `entity_type` и `entity_id` (что изменил), а также `old_value` и `new_value` в формате JSON. Это позволяет полностью восстановить историю изменений любой карточки или доски.

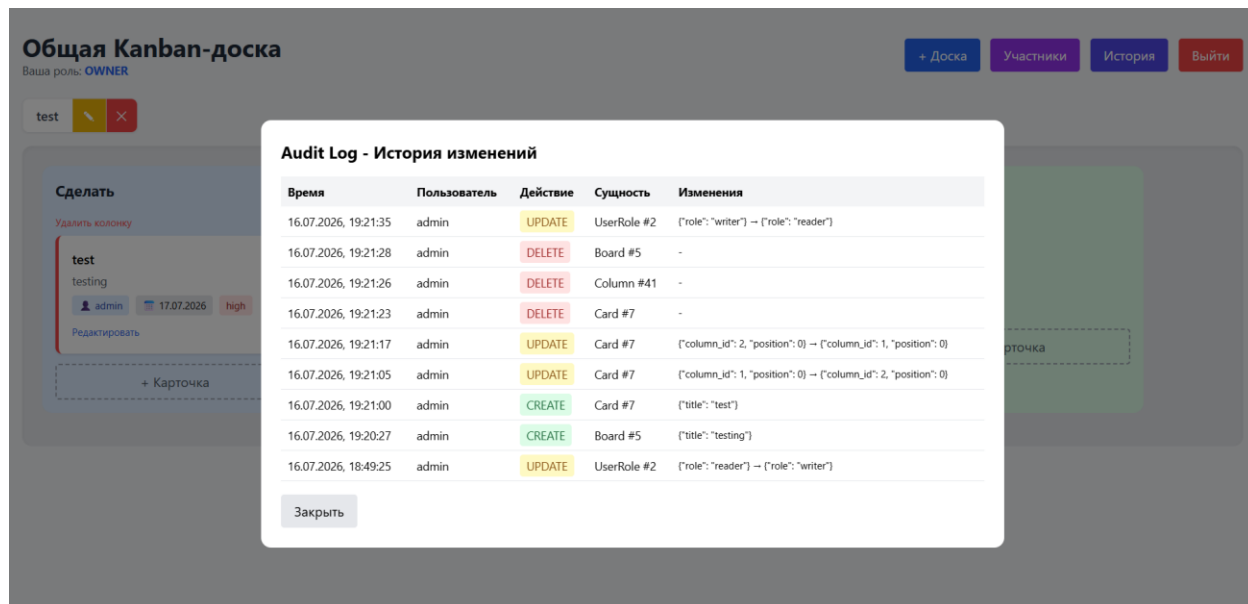


Рисунок 8. Audit Log на сайте

Защита от коллизий:

В модели Card предусмотрено поле version (Integer, default=1). Это архитектурный задел для реализации паттерна Optimistic Locking. В текущей MVP-версии целостность данных при одновременном редактировании гарантируется механизмом транзакций PostgreSQL (через db.commit() в SQLAlchemy), что предотвращает базовые состояния гонки (race conditions).

8. ЗАКЛЮЧЕНИЕ

В ходе летней практики было успешно разработано веб-приложение «Kanban-доска», полностью соответствующее техническому заданию.

Достигнутые результаты:

- Реализован полный CRUD для досок, колонок, карточек и комментариев.
- Внедрен drag-and-drop интерфейс с сохранением позиций на сервере.
- Настроена безопасная аутентификация (JWT + bcrypt) и ролевая модель RBAC.
- Реализован полноценный Audit Log с сохранением старых и новых значений.

- Написаны оптимизированные SQL-запросы с использованием JOIN и GROUP BY.

Проект размещен в репозитории GitHub, код структурирован, документирован и готов к демонстрации.

ИСТОЧНИКИ

GitHub: <https://github.com/mrkvrn/Kanban>

PostgreSQL Documentation (Transactions):

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

SortableJS (Drag and Drop): <https://sortablejs.github.io/Sortable/>

Приложение А. Инструкция по запуску проекта

- Установите зависимости: `pip install -r requirements.txt`
- Убедитесь, что PostgreSQL запущен, и создайте базу данных (или используйте существующую postgres).
- Настройте строку подключения в файле `database.py` (по умолчанию: `postgresql://postgres:postgres@localhost:5432/postgres`).
- Инициализируйте таблицы и создайте администратора: `python init_db.py`
- Запустите сервер: `uvicorn main:app --reload`
- Откройте в браузере: `http://localhost:8000`